

Document Number: P0653r0

Date: 2017-05-28

Project: Programming Language C++

Audience: Library Evolution Working Group

Author: Glen Joseph Fernandes (glenjofe@gmail.com)

pointer_traits utility to convert to raw pointer

This paper proposes adding a new static member function to class template `pointer_traits` (called `to_address`) to obtain a raw pointer from an object of any pointer-like type.

Motivation

It is often necessary to obtain a raw pointer from an object of any pointer-like type. One common use is writing allocator-aware code where an allocator's `pointer` member type is not a raw pointer type.

Typically the expression `addressof(*p)` is used but this is not well-defined when `p` does not reference storage that has an object constructed in it. This means that using this expression to obtain a raw pointer for the purpose of constructing an object (e.g. via a placement *new-expression* or via an allocator) is incorrect.

A typical example of such code:

```
P p = a.allocate(n);
new (addressof(*p)) T();
```

The correct code now looks like:

```
P p = a.allocate(n);
new (pointer_traits<P>::to_address(p)) T();
```

Why pointer_traits?

Typically implementors work around this problem by defining a utility like the following:

```
template <class Ptr>
typename pointer_traits<Ptr>::element_type* to_address(Ptr p) noexcept
{
    return to_address(p.operator->());
}

template <class T>
T* to_address(T* p) noexcept
{
    return p;
}
```

The C++ standard library already provides `pointer_traits` for supporting any pointer-like types and this `to_address` is the natural inverse of its `pointer_to` member function.

Implementation

The [Boost](#) C++ library collection now contains an implementation of this proposal in the Core library. That `boost::pointer_traits` implementation is used by several Boost libraries, starting with the 1.65 release.

Proposed Wording

All changes are relative to [N4640](#).

1. Insert into 20.10.3 [pointer.traits] as follows:

```
namespace std {
    template <class Ptr> struct pointer_traits {
        using pointer = Ptr;
        using element_type = see below;
        using difference_type = see below;

        template <class U> using rebind = see below;
    };
}
```

```

static pointer pointer_to(see below r);
static element_type* to_address(pointer p) noexcept;
};

template <class T> struct pointer_traits<T*> {
    using pointer = T*;
    using element_type = T;
    using difference_type = std::ptrdiff_t;

    template <class U> using rebind = U*;

    static pointer pointer_to(see below r) noexcept;
    static element_type* to_address(pointer p) noexcept;
};
}

```

2. Add to 20.10.3.1 [pointer.traits.functions] as follows:

```

static element_type* pointer_traits::to_address(pointer p) noexcept;
static element_type* pointer_traits<T*>::to_address(pointer p) noexcept;

```

Returns: A pointer of type `element_type*` that references the same location as the argument `p`.

Remarks: The first member function returns `pointer_traits<E>::to_address(expr)` where *expr* is `p.operator->()` and *E* is its type. The second member function returns `p`.

Acknowledgments

Peter Dimov suggested a better design as well as reviewed the implementation and tests of the Boost version. Jonathan Wakely pointed out the issue that motivated this proposal. Michael Spencer reviewed this paper.

References

- N4640, Working Draft, Standard for Programming Language C++,
<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/n4640.pdf>